

Chương 8 : Độ phức tạp

8.1. Định nghĩa :

- Các lệnh cơ bản :

- $x=y+z$, $x=y*z$, $x=-y$, ..., $x=y \ T \ z$, T là phép toán 2 ngôi,
- `printf`, `scanf`, ..., các lệnh nhập xuất,
- Các phép toán so sánh : $<$, $<=$, $>$, $>=$, $==$, $!=$,
- Các phép nối : $\&\&$, $||$, $!$ (phủ định). $A \ \&\& \ B$ được xem là một lệnh cơ bản, với A , B không có các phép so sánh.

VD 1: $x=y+z-t$, gồm các lệnh cơ bản :

$v=y+z$;

$x=v-t$;

VD 2: $(x+y > y+z) \ \&\& \ (x!=1)$, gồm các lệnh cơ bản :

$x+y$, $y+z$, $x+y > y+z$, $x!=1$, $A \ \&\& \ B$, với $A \equiv x+y > y+z$, $B \equiv x!=1$.

8.2. Định nghĩa độ phức tạp:

Độ phức tạp của một thuật toán là số lệnh cơ bản trong thuật toán, ký hiệu T.

VD 1 : Cho thuật toán

```
scanf("%d", &y, &z, &t); (1)
```

```
x=y+z-t;
```

```
printf("%d", x);
```

Phân tích như slide trước , $T = 5$.

Lưu ý : Lệnh (1) gồm 3 lệnh cơ bản. Ta sẽ qui ước là 1 lệnh cơ bản. Tương tự cho printf.

VD 2 : Cho thuật toán

```
scanf("%d", &x, &y, &z);
```

```
if ((x+y > y+z) && (x!=1)) printf("%d", x);
```

Phân tích như slide trước , các lệnh cơ bản

$x+y$, $y+z$, $x+y > y+z$, $x \neq 1$, $A \ \&\& \ B$, với $A \equiv x+y > y+z$, $B \equiv x \neq 1$, scanf, printf, T=7.

Lưu ý : *if (C) Lệnh 1; else Lệnh 2;*

$T = \text{Số lệnh cơ bản } C + \text{Số lệnh cơ bản Lệnh 1} + \text{Số lệnh cơ bản Lệnh 2}$

(Cách tính khác

$T = \text{Số lệnh cơ bản } C + \text{Max}\{ \text{Số lệnh cơ bản Lệnh 1}, \text{Số lệnh cơ bản Lệnh 2} \}$)

VD 3 : Cho thuật toán

```
scanf("%d", &n); T=0;
```

```
i=1;
```

```
while (i<=n) {
```

```
    T=T+i; i=i+1;
```

```
}
```

```
printf("%d", T);
```

- Vòng lặp while thực hiện $T=T+i; i=i+1;$ n lần, số lần của nó là $2n$,
- Lệnh $i \leq n$ thực hiện $n+1$ lần (?),
- Các lệnh còn lại số lần là 4,

Vậy $T=T(n) = 2n+n+1+4 = 3n+5$.

Định nghĩa $O(g(n))$: Cho $f, g : \mathbb{N}^* \rightarrow \mathbb{N}^*$, $\mathbb{N}^* = \{1, 2, \dots\}$. Ta viết

$$f(n) = O(g(n)),$$

(Đọc là $f(n)$ **big oh** của $g(n)$)

nếu có hằng số $C > 0$ sao cho $|f(n)| \leq C|g(n)|$.

VD 1 : $f(n) = 60n^2 + 5n + 1 \leq 60n^2 + 5n^2 + n^2 = 66n^2$, $n \in \mathbb{N}^*$. $C=66$, vậy

$$f(n) = O(n^2).$$

VD 2 : Trong VD 3, slide trước, ta có độ phức tạp của thuật toán là

$$T(n) = 2n + n + 1 + 4 = 3n + 5 \leq 3n + 5n = 8n. \quad C=8, \quad T(n) = O(n).$$

Ý nghĩa : Khi nói độ phức tạp của một thuật toán $T(n) = O(g(n))$ có nghĩa là với giá trị n lớn thì số lệnh cơ bản trong thuật toán xấp xỉ $Cg(n)$.

Các qui tắc trên $O(g(n))$:

Quy tắc bỏ hằng số

Nếu đoạn chương trình P có thời gian thực hiện $T(n) = O(c.f(n))$ với c là hằng dương thì có thể coi đoạn chương trình đó có độ phức tạp tính toán là $O(f(n))$.

Quy tắc lấy max

Nếu đoạn chương trình P có thời gian thực hiện $T(n) = O(f(n)+g(n))$ thì có thể coi đoạn chương trình đó có độ phức tạp tính toán là $O(\max(f(n),g(n)))$.

Các qui tắc trên $O(g(n))$:

Quy tắc cộng

Nếu đoạn chương trình P_1 có thời gian thực hiện $T_1(n) = O(f(n))$ và đoạn chương trình P_2 có thời gian thực hiện $T_2(n) = O(g(n))$ thì thời gian thực hiện P_1 rồi đến P_2 là $T_1(n) + T_2(n) = O(f(n) + g(n))$

Quy tắc nhân

Nếu đoạn chương trình P có thời gian thực hiện $T(n) = O(f(n))$. Khi đó nếu thực hiện $k(n)$ lần chương trình P với $k(n) = O(g(n))$ thì độ phức tạp tính toán là $O(f(n).g(n))$.

8.3 Cải tiến giải thuật : Cải tiến giải thuật là làm cho thuật toán có độ phức tạp nhỏ nhất.

VD 1 : Xét 2 chương trình tính tổng từ 1 đến n

❖ Chương trình 1:

```
scanf("%d",&n);  
S = 0;  
for (int i=1;i<=n;i++)  
    S = S + i;  
printf("%d",S);
```

❖ Chương trình 2:

```
scanf("%d",&n);  
S = n*(n+1)/2;  
printf("%d",S);
```

VD 2 : Xét 2 chương trình tính tổng từ 1 đến n

$$e^x \approx 1 + \frac{x}{1!} + \frac{x^2}{2!} + \dots + \frac{x^n}{n!} = \sum_{i=0}^n \frac{x^i}{i!} \text{ với } x \text{ và } n \text{ cho trước.}$$

❖ Chương trình 1: Tính riêng từng hạng tử rồi cộng lại

```
scanf("%f%d",&x,&n);  
S = 0;  
for (int i=0;i<=n;i++) {  
    p = 1;  
    for (int j=1;j<=i;j++) p = p*x/j;  
    S = S + p;  
}
```

❖ Chương trình 2: Tính hạng tử sau qua hạng tử trước

```
scanf("%f%d",&x,&n);  
S = 1; p = 1;  
for (int i=1;i<=n;i++) {  
    p = p*x/i;  
    S = S + p;  
}
```